김수환 · 웹 개발자 · GitHub ↗

포트폴리오

프로젝트별 담당 업무, 문제 해결 과정과 구현 자료입니다.

다온소프트랩 · 업무 프로젝트

Bitria

멀티 거래소 연동 암호화폐 거래 플랫폼

2024.08 – 현재 | 프론트엔드·백엔드 API 개발 담당

여러 암호화폐 거래소를 연동해 현물·선물·마진 거래를 제공하는 플랫폼입니다. 웹·모바일 앱의 프론트엔드와 백엔드 API 개발을 담당하고, 인프라 운영에 참여했습니다.

주요 업무

거래 화면	TradingView 차트, 실시간 시세·호가·체결, 현물·선물·마진 주문 및 포지션·체결 내역 구현
백엔드 API	주문 생성·조회·취소·수정, 요청 검증·권한 확인·주문 처리 서비스 연동. 계좌별 자산·USDT 환산·자산 스냅샷·일별 손익 조회 구현
인증·보안	회원가입·로그인·소셜 로그인, 2FA·패스키·신뢰 기기·세션 관리·계정 복구 API와 화면 개발. KYC·입출금 서비스 연동 및 API 키 관리
멀티 플랫폼	동일 코드베이스에서 웹·iOS·Android와 Electron 데스크톱 빌드 대응. 모바일 앱 스토어 출시, 네이티브 푸시·소셜 로그인·생체인증 연동
공통 구조	FSD 아키텍처와 시맨틱 디자인 토큰 설계. 다크·라이트 테마, 브랜드 2종·6개 로케일 지원. 웹·앱 차이에 따른 폴더·props·slot 사용 기준 문서화
서비스 운영	가격 조건별 알림·클라이언트 에러 수집·앱 버전 정책·CS 티켓 API 개발. k3s 프로덕션 배포·운영 및 이슈 대응

기술 TypeScript · Next.js 16 · React 19 · Tailwind CSS · Zustand · TanStack Query · Capacitor · Electron · Node.js · Express · Prisma · MariaDB · Redis · Docker · k3s

대표 문제 해결

주문 가능액 계산 불일치 개선

레버리지 모달과 주문 폼이 최대 주문 가능액을 각각 계산해, 같은 조건에서 약 56배의 표시 차이가 발생했습니다. 거래소 공식 문서와 실제 API 응답을 대조해 교차 마진에 격리 마진 전용 리스크리밋을 적용한 오류와 잔고×레버리지 단순 계산을 원인으로 확인했습니다.

표시값만 수정하면 계산 규칙이 바뀔 때 다시 어긋날 수 있어, 수수료·안전 버퍼·리스크리밋·레버리지 상한·최소 주문 단위를 반영하는 공통 계산 모듈로 통합했습니다. 주문 수량의 부동소수점 오차는 BigNumber 기반으로 처리했습니다.

> **결과**
> 두 화면의 계산값을 랏 단위까지 일치시켰습니다. 경계 사례 20건을 검증하고, 운영 환경 확인 과정에서 발견한 양방향 포지션의 조건부 요청 누락도 수정했습니다.

모바일 홈의 스크립트 실행 비용 개선

프로덕션 모바일 홈 진단에서 Lighthouse 38점·TBT 1,610ms를 확인했습니다. 청크별 실행 비용을 분석한 결과, shared/ui의 일괄 재수출 경로가 불필요한 모듈을 같은 청크로 묶고 있었습니다.

optimizePackageImports 적용은 청크 수와 전체 번들을 늘려 되돌리고, 컴포넌트별 직접 경로 import와 three.js 배경 지연 로딩을 적용했습니다.

> **결과**
>
> 로컬 프로덕션 빌드의 전후 비교에서 TBT 443 → 226ms(약 49% 감소), 초기 JavaScript 전송량 22KB 감소를 확인했습니다. 전체 번들 증가분은 초기 로드에 포함되지 않음을 확인했습니다.

Lighthouse 모바일 프리셋, CPU 4배 스로틀·느린 4G, 워밍업 후 3~5회 측정. 프로덕션 진단값 1,610ms와 로컬 전후 비교값은 서로 다른 환경의 측정입니다.

실시간 갱신 범위를 표시 컴포넌트로 제한

실시간 스냅샷을 페이지 최상위 상태에서 받아 시세가 갱신될 때 주문 폼·하단 패널·닫힌 UI까지 함께 렌더링됐습니다. 웹 화면은 같은 문제가 없음을 측정으로 확인했습니다.

앱 라우트에 한정해 구독을 표시 컴포넌트로 내리고, props와 구독 인자의 참조를 안정화했습니다.

> **결과**
>
> 12초 프로파일링에서 누적 렌더 시간 1,028 → 416ms(약 60% 감소), 렌더 횟수 2,802 → 1,323회(약 53% 감소)를 확인했습니다.

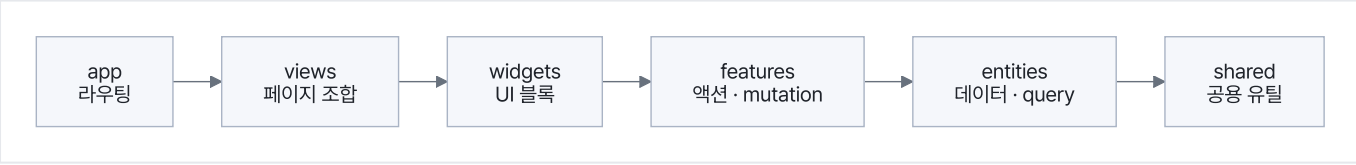
앱 현물 트레이딩 화면에서 조작 없이 실시간 데이터만 갱신한 구간입니다. 해당 구간에서 주문 폼·하단 패널의 불필요한 렌더링도 줄었습니다.

구현 구조와 설계

FSD 아키텍처와 화면 분기 기준

화면과 도메인이 늘어날 때 의존성이 얽히지 않도록 app → views → widgets → features → entities → shared의 단방향 의존 규칙을 세웠습니다. 조회(query)는 entity에, 변경(mutation)은 feature에 두어 레이어 책임을 분리했습니다.

웹·앱의 라우트가 다르면 폴더를 분리하고, 뷰포트 차이가 작으면 prop 분기, 마크업 차이가 크면 디바이스별 파일을 두는 기준을 정했습니다.

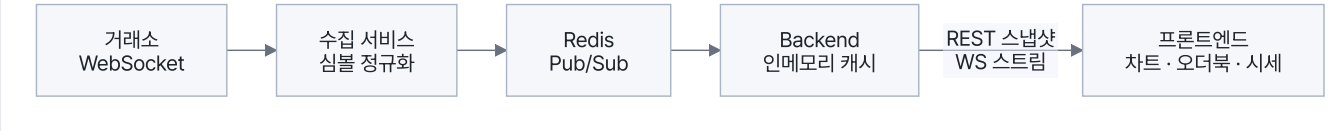


FSD 레이어 간 의존 방향과 각 레이어의 책임

실시간 데이터 전달과 구독 구조

거래소 WebSocket 데이터는 수집 서비스의 심볼 정규화와 Redis Pub/Sub을 거쳐 백엔드에 전달됩니다. 프론트엔드는 REST 스냅샷과 WebSocket 스트림으로 차트·호가·시세를 표시합니다.

실시간 구독은 값이 실제로 쓰이는 표시 컴포넌트로 내리고, 티커도 항목별로 구독하도록 분리했습니다. 가려진 오더북과 닫힌 드로어의 구독을 중지하고 체결 행에는 memo를 적용해 갱신 범위를 줄였습니다.

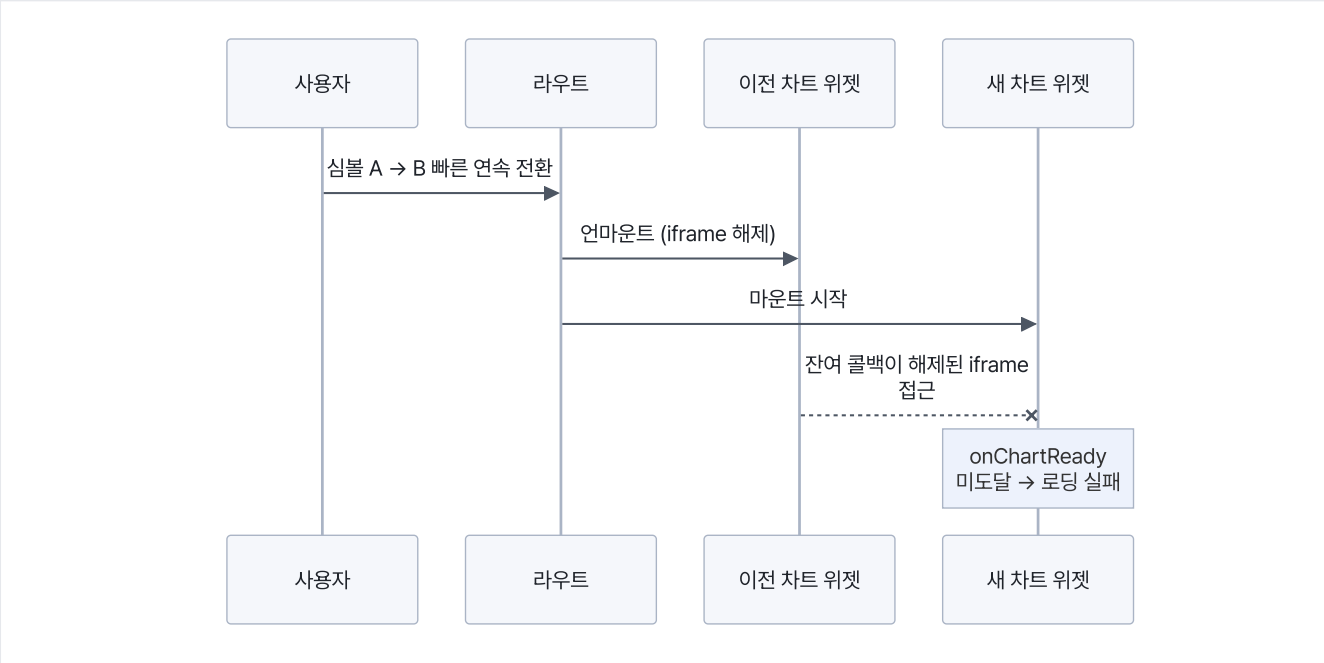


실시간 데이터가 프론트엔드에 전달되는 흐름

차트 로딩 실패의 원인 분석과 해결

간헐적인 차트 로딩 실패를 클라이언트 에러 로그와 서버·CDN 응답 시간으로 추적했습니다. 심볼을 빠르게 연속 전환하면 이전 위젯의 잔여 콜백이 이미 해제된 iframe에 접근하면서 새 위젯이 준비 상태에 도달하지 못하는 경쟁 상태를 재현했습니다.

라우트 파라미터가 바뀔 때 페이지 서브트리가 통째로 리마운트되는 구조가 원인이었습니다. 차트 뷰를 레이아웃 레벨로 올려 리마운트를 없애고, 위젯이 준비되기 전에 변경된 심볼을 보정하는 처리를 추가했습니다.

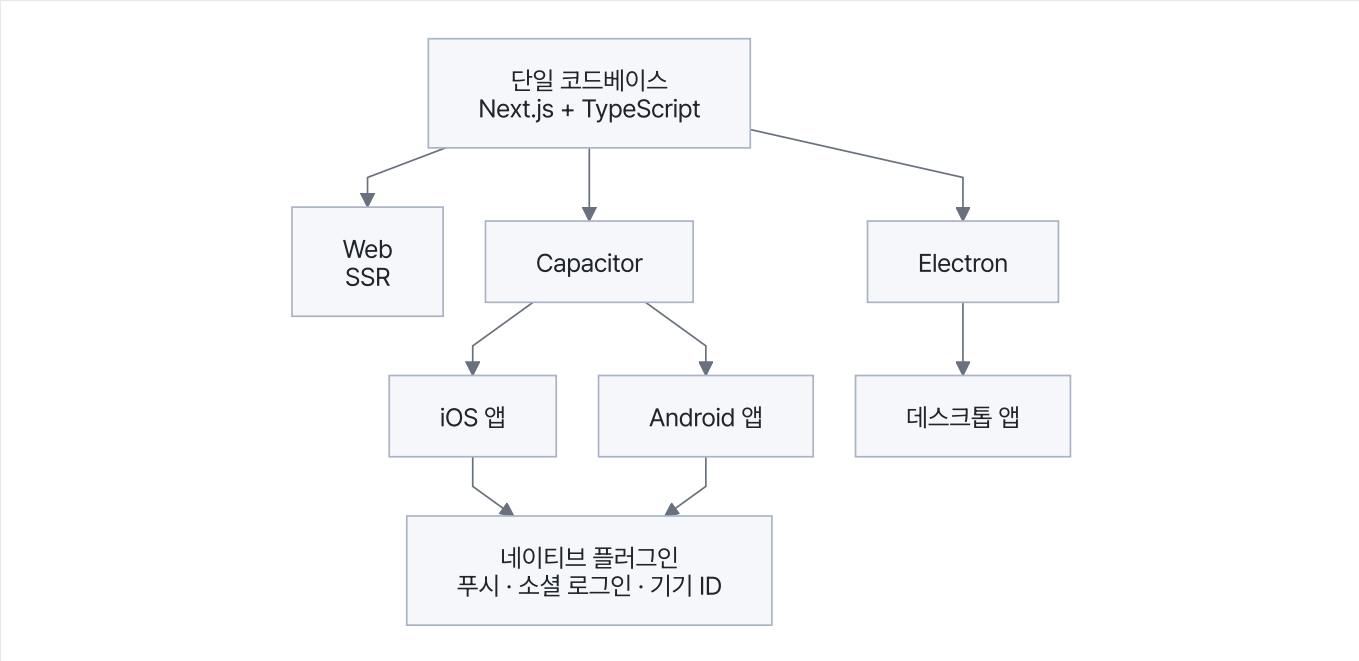


수정 전: 빠른 심볼 전환에서 발생한 위젯 생명주기 충돌

단일 코드베이스의 멀티 플랫폼 대응

Next.js·TypeScript 코드베이스를 웹, Capacitor 기반 iOS·Android 앱, Electron 데스크톱 앱으로 빌드했습니다. 모바일 앱은 스토어에 출시하고 푸시(APNs·FCM), 소셜 로그인, 기기 식별 플러그인을 연결했습니다.

앱 강제 업데이트 버전 정책을 연동하고, Electron 요청을 구분하는 헤더 분기를 적용했습니다.



웹·모바일·데스크톱 빌드와 네이티브 기능 연결 구조

다온소프트랩 · 업무 프로젝트

gobit

코인 정보·커뮤니티 플랫폼

2024.10 – 현재 | 프론트엔드·백엔드 개발 담당 · 2인 팀

암호화폐 시세·차트·뉴스·커뮤니티와 RSM 모의투자 연계를 제공하는 서비스입니다. 2인 팀에서 웹 클라이언트를 신규 구축하고 백엔드 API를 함께 개발했습니다.

주요 업무

웹 클라이언트	Next.js 15·React 19 기반 마켓·심볼 상세·뉴스·커뮤니티·랭킹·마이페이지 구현. TradingView 차트와 tiptap 에디터 연동
실시간 데이터	시세는 Socket.IO, 그 외 조회는 SWR로 관리. Binance 데이터 수집·Redis 캐시·소켓 재연결·헬스체크 개선
서비스 기능	회원 인증·출석·레퍼럴·게시글·댓글·북마크 등 서비스 기능과 API 개발
운영 개선	SSR·이미지·폰트·SEO·LCP·심볼 페이지 렌더링 개선. coinIns → gobit 리브랜딩, 점검 모드·회원 탈퇴 구현

> **결과**
> 웹 클라이언트 전 페이지를 구축해 서비스에 적용하고, 리브랜딩과 운영 기능을 포함한 개발 후 유지보수를 이어갔습니다.

기술 TypeScript · Next.js 15 · React 19 · SWR · Zustand · Radix UI · TradingView · tiptap · Node.js · Express · Prisma · Redis · Socket.IO

대표 문제 해결

레거시 플랫폼 통합과 Kubernetes 이전

2026.07 – 2026.08 · 이전 후 유지보수

gobit·RSM·시그널이 Windows·Linux 서버 5대에 분산돼 있었고, 핵심 시세 수집기는 소스가 유실된 .NET 바이너리만 남아 있었습니다. 유지보수와 배포를 이어갈 수 있도록 서비스들을 모노레포로 통합하고 Kubernetes 환경으로 이전했습니다.

- 기존 바이너리의 동작과 데이터 형식을 분석해 수집기를 Node.js로 재작성. 실시간·캔들·백필 모드 구현, 연결 샤딩·스로틀로 WebSocket 끊김 대응
- 중계 계층을 정리해 수집기 → Redis → 서버 구조로 단순화
- 13개 서비스의 Dockerfile·Kubernetes 매니페스트, Ingress·DB 백업·캔들 보관정책 CronJob 작성
- GitHub Actions 빌드·배포 파이프라인 구성 및 Kubernetes Secret 참조 전환

> **결과**
> 서버 5대에 분산된 플랫폼을 모노레포·Kubernetes 환경으로 이전하고, 재작성한 수집기를 기존 동작과 대조 검증했습니다. 13개 서비스의 빌드·배포를 자동화했습니다.

두잇 · 업무 프로젝트

doit-uikit

사내 공통 UI 라이브러리

2024.02 – 2024.07 | 프론트엔드 개발 담당

프로젝트마다 반복 구현하던 UI를 공통 라이브러리로 개발하고, React 개발 규칙을 팀에 공유했습니다.

주요 업무

컴포넌트	Atomic Design 기반 UI 컴포넌트 24종 개발. 기본 요소 12종과 차트 5종을 포함한 복합 요소 12종 구성
패키징	React·React DOM을 peer dependency로 분리하고 vite-plugin-dts로 타입 선언을 자동 생성해 npm 배포
문서화	Storybook·Chromatic으로 사용법·Props·상태별 UI 문서화
팀 표준	디렉토리 구조·디자인 패턴·라이브러리 선정 기준 정립, ESLint·Prettier 설정 및 팀 온보딩 교육

> **결과**
> D.A.X와 회사 홈페이지 2개 프로젝트에 도입해 요구사항에 맞춰 버전을 올리며 유지보수했습니다. 문서·린터 설정·실제 라이브러리를 함께 제공해 공통 UI와 개발 규칙을 제품에 적용했습니다.

기술 TypeScript · React 18 · Vite · styled-components · Storybook 8 · Chromatic · vite-plugin-dts

---

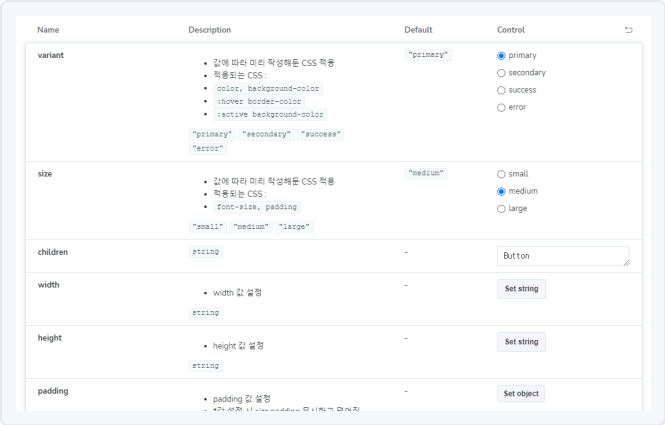
구현 구조와 설계

컴포넌트 조합과 라이브러리 배포

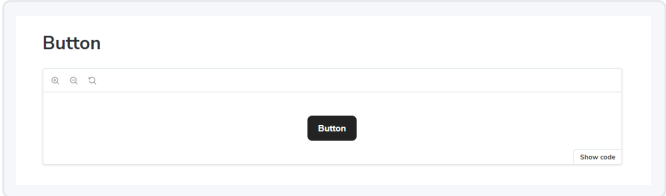
Button·Input 같은 Atom을 조합해 FileInput 등의 Molecule을 구성했습니다. Storybook에서 Props·상태·사용 예를 확인하고 Playground로 직접 값을 바꿔볼 수 있도록 문서화했습니다.

vite-plugin-dts로 타입 선언을 함께 배포하고 React·React DOM은 peer dependency로 분리했습니다. D.A.X와 회사 홈페이지에 적용하면서 발견한 요구사항을 라이브러리에 반영했습니다.

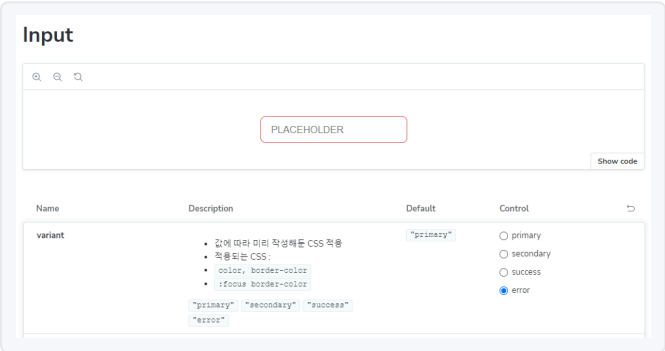
doit-uikit · 프로젝트 화면



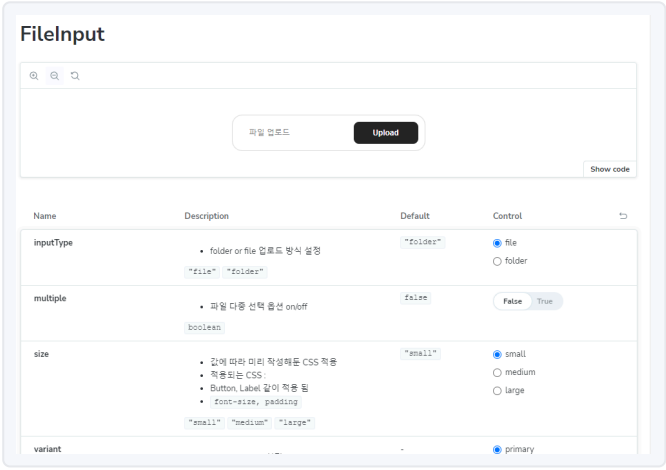
doit-uikit 컴포넌트의 Storybook 문서와 Props 설정 화면



Button 컴포넌트의 Storybook 예시



Input 컴포넌트의 상태·Props 문서



Input·Button·Label을 조합한 FileInput 컴포넌트

npm 패키지 ↗

두잇 · 업무 프로젝트

D.A.X

AI 모델 학습·운영 플랫폼

2024.01 – 2024.06 | 프론트엔드 개발 담당 · 백엔드 1명과 협업

데이터 수집·라벨링부터 모델 학습·추론까지 지원하는 사내 플랫폼입니다. 백엔드 개발자 1명과 협업해 프론트엔드 전 화면을 개발했습니다.

주요 업무

플랫폼 화면 프로젝트·대시보드·클래스·컬렉션·라벨링·모델 학습·추론 화면 구현

데이터 도구 캔버스 바운딩 박스 라벨링 도구와 이미지·폴더 업로드, 컬렉션 ZIP 다운로드 개발

공통 구조 검색 조건·페이지네이션·로딩 상태 처리, 사내 표준 구조와 doit-uikit 적용

> 라벨링 이미지는 화면 크기에 맞춰 축소되지만 학습용 좌표는 원본 해상도로 저장해야 했습니다. 원본↔화면 좌표의 양방향 변환을 전용 훅으로 분리해 그리기·저장·재표시가 같은 규칙을 사용하도록 구현했습니다.

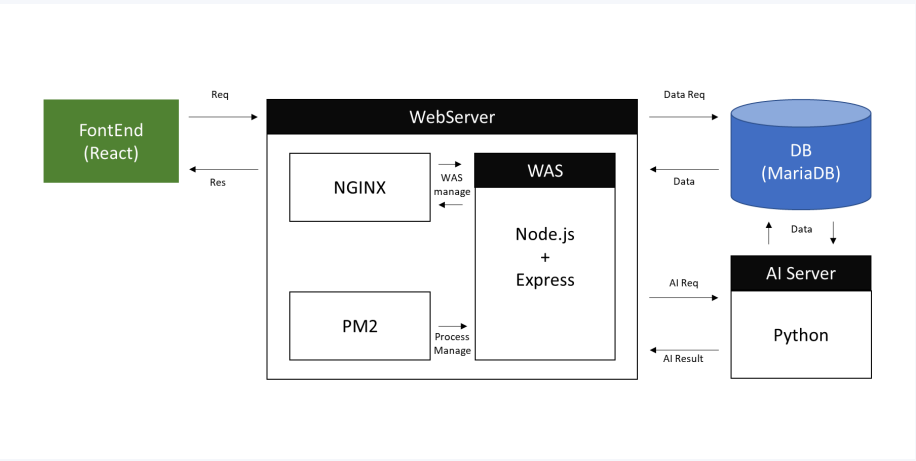
기술 TypeScript · React 18 · TanStack Query 5 · Zustand · React Router 6 · Axios · doit-uikit

구현 구조와 설계

서비스 구성과 프론트엔드 담당 범위

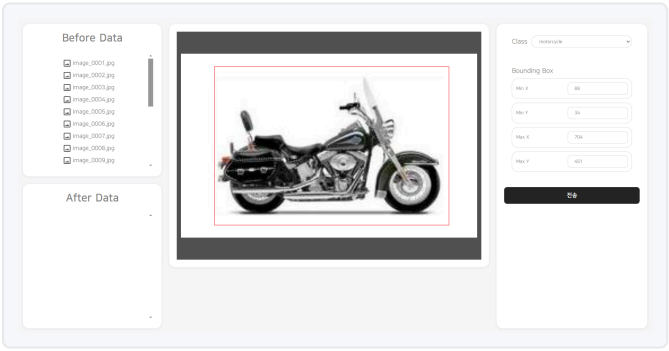
React 프론트엔드는 웹 서버의 API를 통해 프로젝트·라벨·모델 데이터를 조회하고 변경합니다. 백엔드와 AI 서버가 연결된 전체 구성 중 프론트엔드 화면과 데이터 연동을 담당했습니다.

사내 표준 디렉토리 구조를 적용하고 API 호출을 auth·project·class·datamanage·aiModel·operation 등 도메인별 서비스 모듈로 나눴습니다. 원본·렌더 좌표 변환은 useCoordinateTransform 훅으로 분리해 그리기·저장·재표시에 같은 규칙을 적용했습니다.

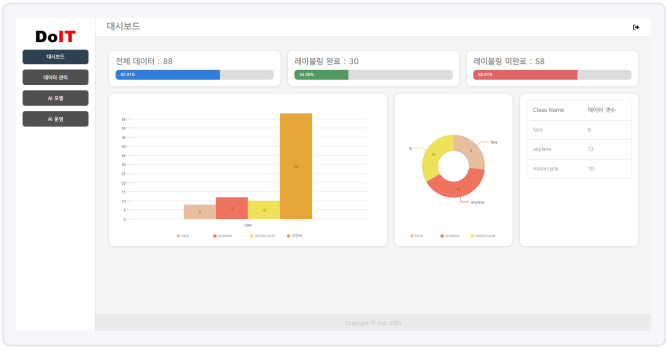


D.A.X 전체 서비스 구성 — 담당 범위는 React 프론트엔드와 API 연동

D.A.X · 프로젝트 화면



D.A.X에서 이미지 위에 바운딩 박스를 그려 라벨링하는 화면



D.A.X의 프로젝트 데이터와 학습 현황 대시보드

두잇 · 업무 프로젝트

doit-homepage

회사 홈페이지 React 마이그레이션

2024.05 – 2024.07 | 프론트엔드 개발 담당

jQuery 기반 회사 홈페이지를 React·TypeScript·Vite 기반 SPA로 재작성하고 다국어·반응형에 대응했습니다.

주요 업무

마이그레이션 메인·회사소개·서비스·인증·파트너 5개 페이지를 컴포넌트 구조로 재작성

UI 재사용 사내 UI 라이브러리 doit-uikit을 적용해 색상·타이포·레이아웃 등 공통 요소 재사용

다국어 i18next 기반 한국어·영어 지원. 번역 리소스를 페이지별 네임스페이스로 분리

반응형 CSS·JavaScript가 공통 브레이크포인트를 사용하도록 구성. 치수 변화는 CSS로 처리하고 모바일에서 불필요한 영역은 렌더링 생략

> **결과**
> 주요 페이지의 React 전환과 2개 언어·반응형 대응을 완료했습니다. 공통 라이브러리를 적용하고, 사용 과정에서 필요한 개선을 라이브러리에 반영했습니다.

기술 TypeScript · React 18 · Vite · styled-components · doit-uikit · i18next · react-responsive · Zustand · React Router 6 · Kakao Maps SDK

두잇 · 업무 프로젝트

XRGIS BackOffice

XR·GIS 관리자 도구

2023.08 – 2023.12 | 프론트엔드·백엔드 개발 담당

XR·GIS 서비스의 사용자·팀·대용량 파일을 관리하는 사내 백오피스입니다. 프론트엔드·백엔드와 데이터베이스 연동을 개발했습니다.

주요 업무

풀스택 개발 React 기반 사용자·팀·데이터 관리 화면과 Express API 개발

파일 처리 MongoDB GridFS 기반 대용량 파일 업로드, archiver 기반 ZIP 일괄 다운로드 구현

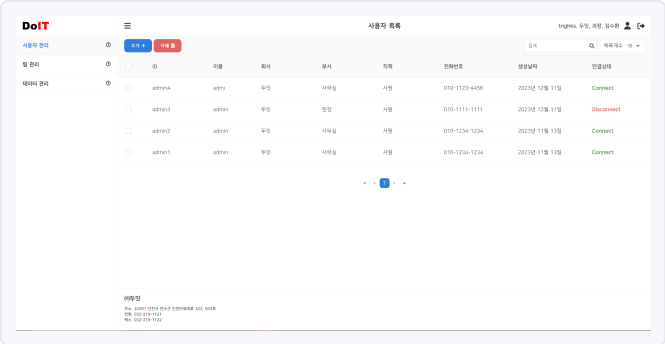
인증·권한 사내 공통 관리자 계정의 세션 인증과 팀 소속에 따른 파일 다운로드 권한 적용

상태·피드백 서버 상태는 React Query, UI 상태는 Recoil로 분리하고 API 결과를 Toast로 안내

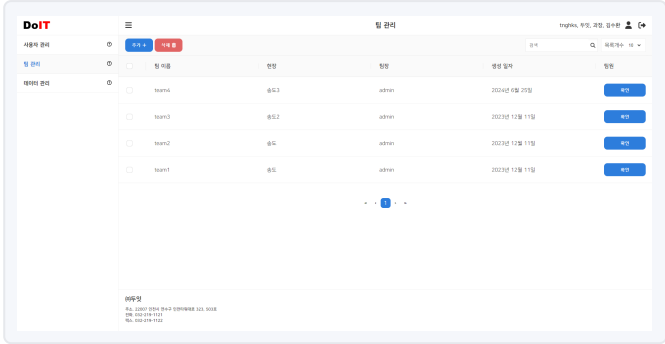
> MongoDB에 파일을 저장해야 하는 요구사항에 맞춰, 일반 문서의 16MB 제한에 걸리는 OBJ·영상 등을 GridFS의 청크 분할 저장으로 처리했습니다. 같은 관리자 계정을 사용하는 사내 서비스와는 MySQL 기반 세션 저장소를 통해 인증 체계를 맞췄습니다.

기술 TypeScript · React 18 · React Query · Recoil · styled-components · Express · MongoDB(GridFS) · MariaDB

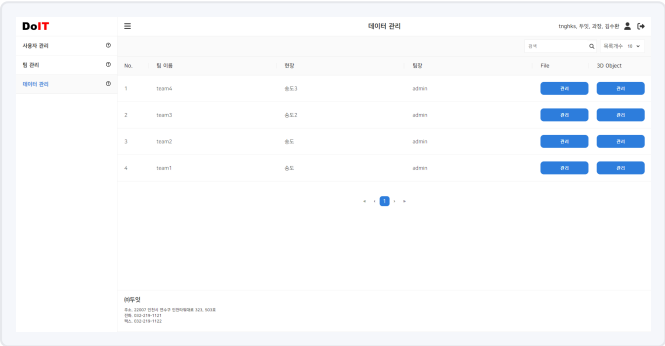
XRGIS BackOffice · 프로젝트 화면



XRGIS 사용자 관리 화면



XRGIS 팀 관리 화면



XRGIS 팀별 데이터·파일 관리 화면